Lctr 8: Exception and Interrupt

Jianhua Liu

Spring 2026

==Topics==

Core concepts and applications:

- What is an interrupt and how it is handled.
- Differences between exception and interrupt.
  - They are for Cortex core and peripherals, respectively.
- NVIC—nested vectored interrupt controller.
- Servicing interrupts with ISRs (interrupt service routines) and callback functions.
- NVIC functions to manage interrupts.

Demo project:

- Blinking an LED with a rate read from UART using the interrupt mode.
- Doing only one thing in the super loop—everything else is handled by interrupts.

Additional reading:

- Setting up UART to use interrupt with CubeMX.
- C functions for UART interrupt operations.

8.1 Introduction

8.1.1 What is an interrupt and how it is handled

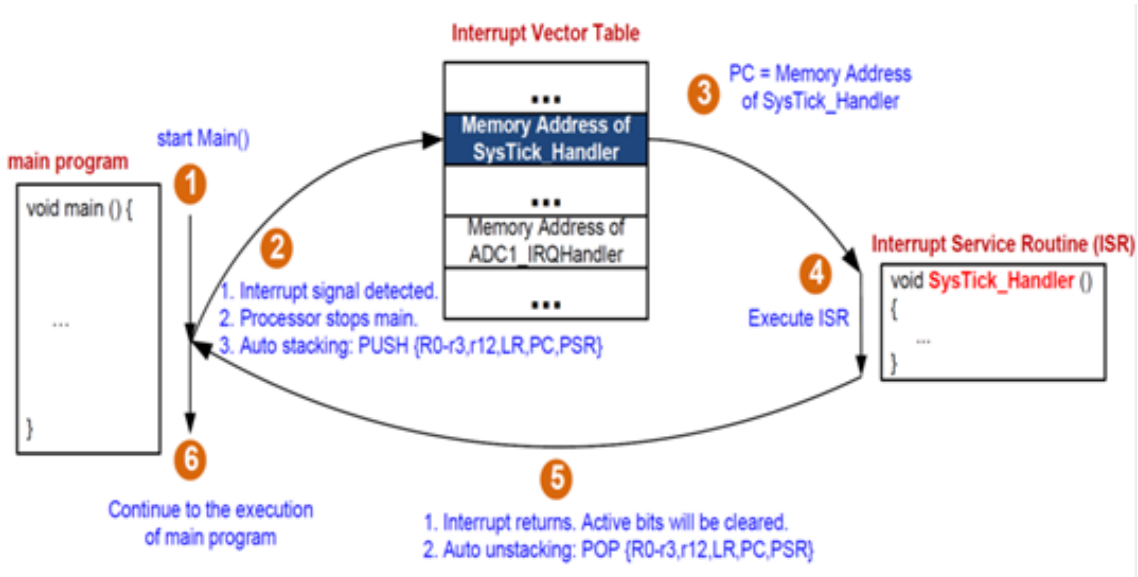

Figure 1: Flow of a program with interrupt.

8.1.2 Importance of interrupt

There are many advantages with using the interrupt-based approach:

- The architecture of program is very simple as compared to that of the polling-based approach, and the program is easier to extend to add more functionalities.
- The response to the event can be much faster than the polling counterpart. This is especially important for the real-time processing of some events.
- The program does not have to run all the time. We can put the processor into sleep to save power consumption. When an interrupt triggers, the processor is woken up to handle the event.

8.2 Demo project

In a previous project, `de2f_direct_gpio_reg_access`, we managed to do two tasks together:

- It blinks an LED at a certain rate.
- It reads user's input from UART using the block mode. Upon reading the correct input, it then reads a GPIO pin and prints out the result.

As the UART works at the block mode in the super loop, it cannot blink the LED in the same loop. As a result, we used a callback function for the SysTick timer to blink the LED.

In this project, we do something similar: blinking an LED with the blinking rate read from UART. Here we do it using a totally different approach:

- Blink the LED at a specified rate in the super loop.
- **Read user input, the blinking rate, from UART using the interrupt mode**.

8.3 NVIC—nested vectored interrupt controller

NVIC is the hardware that makes every aspect of interrupt possible. NVIC has a number of aspects by itself. Let's see how we have the name of NVIC below.

8.3.1 Nested service

8.3.2 Vector table

8.3.3 Interrupt controller

8.3.4 NVIC

Combining the above titles, we then have NVIC.

We have discussed that all aspects of GPIO are controlled by GPIO registers. This is true for NVIC as well. **NVIC has a large number of registers** for controlling how NVIC works.

8.4 Exceptions and interrupts for ARM processors

8.4.1 Exception vs interrupt

- **Exception** is used to mean the *conditions* about the **ARM core** and special software instructions.
- **Interrupt** is used to mean the *expected* conditions about the **peripherals**.

8.4.2 CMSIS IRQn of Cortex-M processors

Cortex-M processors are designed to support up to 255 system exceptions and peripheral interrupts, each having an unique **CMSIS** (Common microcontroller software interface standard) **IRQn** (interrupt request number), for easy programming.

- The first 15 IRQns are **system exceptions**.
  - CMSIS defines the IRQns of system exceptions as *negative numbers* to distinguish them from those of peripheral interrupts.
  - These IRQns are defined by Arm for ARM cores, and hence they are defined for all Cortex-M-based MCUs.
  - They include hardware exceptions (interrupts) about ARM cores, including the SysTick timer and debug monitor. They also include software instructions used to trigger ISRs for special operations.
- The rest IRQns are **peripheral interrupts**.
  - CMSIS defines the IRQns of peripheral interrupts as positive numbers, starting from 0, to distinguish them from those of system exceptions.
  - These IRQns are defined by chip manufactures.

The IRQns for STM32G431 MCUs are defined in `stm32g431xx.h` as an enum. All system exceptions and peripheral interrupts that are mostly interesting to us (which we will discuss later in one way or anther) are listed below:

```
typedef enum {
/******  Cortex-M4 Processor Exceptions Numbers *********/
  NonMaskableInt_IRQn   = -14, // 2 Cortex-M4 Non Maskable
  HardFault_IRQn        = -13, // 3 Cortex-M4 Hard Fault
  MemoryManagement_IRQn = -12, // 4 Cortex-M4 Memory Mngt
  BusFault_IRQn         = -11, // 5 Cortex-M4 Bus Fault Int
  UsageFault_IRQn       = -10, // 6 Cortex-M4 Usage Fault
  SVCall_IRQn           = -5,  // 11 Cortex-M4 SV Call Int
  DebugMonitor_IRQn     = -4,  // 12 Cortex-M4 Debug Mntr
  PendSV_IRQn           = -2,  // 14 Cortex-M4 Pend SV Int
  SysTick_IRQn          = -1,  // 15 Cortex-M4 System Tick
/******  STM32 specific Interrupt Numbers ****************/
  WWDG_IRQn             = 0,   // Window WatchDog Interrupt
  EXTI0_IRQn            = 6,   // EXTI Line0 Interrupt
  EXTI1_IRQn            = 7,   // EXTI Line1 Interrupt
  EXTI2_IRQn            = 8,   // EXTI Line2 Interrupt
  EXTI3_IRQn            = 9,   // EXTI Line3 Interrupt
  EXTI4_IRQn            = 10,  // EXTI Line4 Interrupt
  DMA1_Channel1_IRQn    = 11,  // DMA1 Channel 1 global Int
  DMA1_Channel2_IRQn    = 12,  // DMA1 Channel 2 global Int
  DMA1_Channel3_IRQn    = 13,  // DMA1 Channel 3 global Int
  ADC1_2_IRQn           = 18,  // ADC1 and ADC2 global Int
  USB_HP_IRQn           = 19,  // USB high-priority Int
  USB_LP_IRQn           = 20,  // USB low-priority Int
  EXTI9_5_IRQn          = 23,  // External Line[9:5] Int
  TIM1_CC_IRQn          = 27,  // TIM1 Capture Compare
  TIM2_IRQn             = 28,  // TIM2 global Interrupt
  TIM3_IRQn             = 29,  // TIM3 global Interrupt
  TIM4_IRQn             = 30,  // TIM4 global Interrupt
  I2C1_EV_IRQn          = 31,  // I2C1 Event Interrupt
  I2C1_ER_IRQn          = 32,  // I2C1 Error Interrupt
  I2C2_EV_IRQn          = 33,  // I2C2 Event Interrupt
  I2C2_ER_IRQn          = 34,  // I2C2 Error Interrupt
  SPI1_IRQn             = 35,  // SPI1 global Interrupt
  SPI2_IRQn             = 36,  // SPI2 global Interrupt
  USART1_IRQn           = 37,  // USART1 global Interrupt
  USART2_IRQn           = 38,  // USART2 global Interrupt
  USART3_IRQn           = 39,  // USART3 global Interrupt
  EXTI15_10_IRQn        = 40,  // External Line[15:10] Int
} IRQn_Type;
```

8.4.3 PSR exception number

When an IRQn is saved in a hardware register, called **program status register** (PSR), it is converted to *unsigned* PSR **exception number** by the following approach

```
PSR exception number = 16 + IRQn
```

Figure 2 shows the bit fields of the PSR, where the field for exception number is at bits 0 to 8.

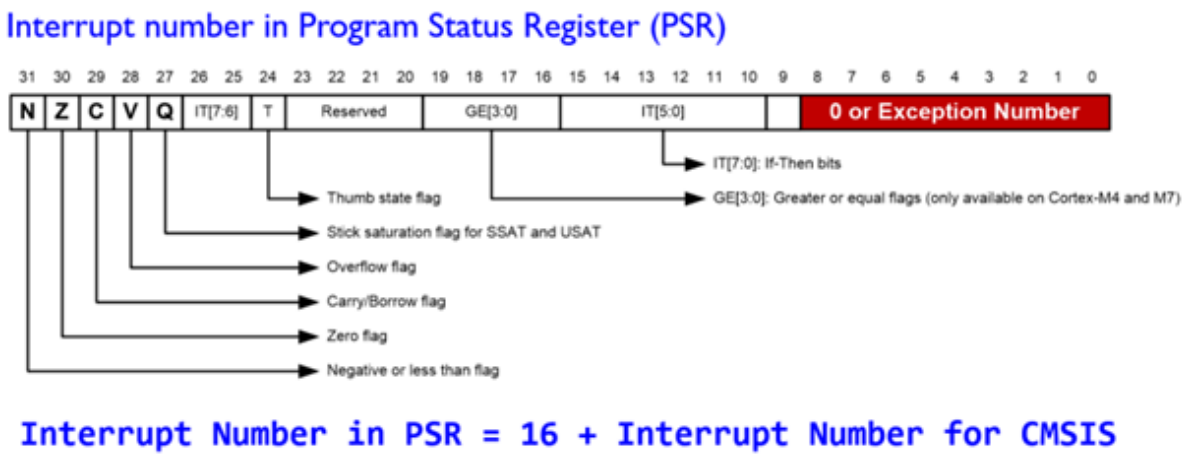

Figure 2: Bit fields of PSR.

Note that when an interrupt occurs, its exception number is put into the PSR, which is used to find the ISR's entry address directly from a vector table, also called NVIC table, as detailed below.

After the interrupt is serviced, its exception number is cleared from PSR.

8.4.4 The NVIC table

This vector table is defined in ASM in `startup_stm32g431kbtx.s`, as shown in the code list below. No need to worry about all the details now. Just pay attention to the definition of IRQn handler names. For example, `SysTick_Handler` which is saved in word 15 in the table, corresponding to `SysTick_IRQn (= -1)+ 16`, will be used to save the entry address of the SysTick timer ISR, as detailed later.

```
/***** NVIC table ******************************************
*
* Minimal vector table for a Cortex-M4.  Note that the proper
* constructs must be placed on this to ensure that it ends up
* at physical address 0x0000.0000.
*
****** NVIC table *****************************************/
  .section  .isr_vector,"a",%progbits
  .type  g_pfnVectors, %object
  .size  g_pfnVectors, .-g_pfnVectors

g_pfnVectors:
  .word  _estack  // Top of stack   0x00
  .word  Reset_Handler
  .word  NMI_Handler         // 0x02 system exception
  .word  HardFault_Handler
  .word  MemManage_Handler   // 0x04
  .word  BusFault_Handler
  .word  UsageFault_Handler  // 0x06
  .word  0
  .word  0                   // 0x08
  .word  0
  .word  0                   // 0x0A
  .word  SVC_Handler
  .word  DebugMon_Handler    // 0x0C
  .word  0
  .word  PendSV_Handle       // 0x0E
  .word  SysTick_Handler
  .word  WWDG_IRQHandler     // 0x10 peripheral interrupt
  .word  PVD_PVM_IRQHandler
  .word  RTC_TAMP_LSECSS_IRQHandler
  .word  RTC_WKUP_IRQHandler
  .word  FLASH_IRQHandler
  .word  RCC_IRQHandler
  .word  EXTI0_IRQHandler
  .word  EXTI1_IRQHandler
  .word  EXTI2_IRQHandler
```

8.4.5 An example of interrupt handler

The start file, `startup_stm32g431kbtx.s`, contains the definitions of the entry points, the start addresses, of the ISR handlers. While many ISR handlers are placeholders, so that the program can compile, the reset handler, `Reset_Handler`, provides the real functioning code.

```
Reset_Handler:
/* set stack pointer */
    ldr   r0, =_estack
    mov   sp, r0

/* Copy the data segment initializers from flash to SRAM */
    ldr r0, =_sdata
    ldr r1, =_edata
    ldr r2, =_sidata
    movs r3, #0
    b  LoopCopyDataInit
```

8.4.6 Interrupt stacking and unstacking

The processor hardware automatically stacks and unstacks a number of registers: R0-R3, R12, LR, PC, and PSR. This way, the programming of the ISRs will be the same as that for a normal function, the caller of which will be saving these registers according to a convention that we will discuss later.

The stacking and unstacking processes are illustrated in Figure 3.

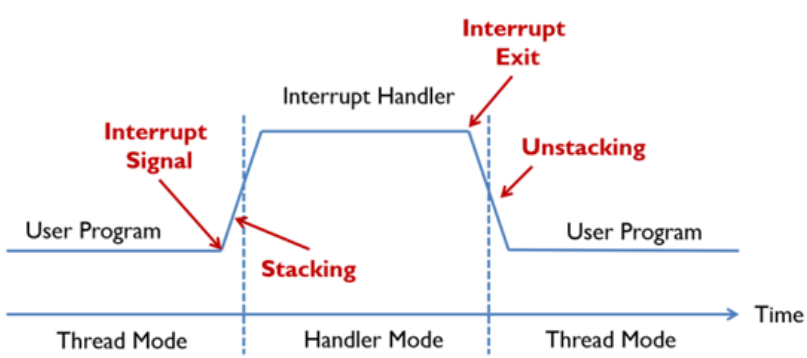

Figure 3: Interrupt stacking and unstacking.

8.5 Managing exceptions and interrupts in C using CMSIS NVIC functions

- `NVIC_EnableIRQ(IRQn)`. This is used when we want to enable interrupt `IRQn`. This happens at the initialization phase or when we want to enable an interrupt again after it is being disabled before.
- `NVIC_DisableIRQ(IRQn)`. This is used when we want to disable interrupt `IRQn`. This happens when we want to disable an interrupt after it is being enabled before.
- `NVIC_ClearPendingIRQ(IRQn)`. This is used to clear the pending status of an `IRQn`. This happens when we do not have capacity to handle the low priority interrupts due to the heavy load of the other interrupts.

8.6 Important NVIC registers

- Interrupt set enable register (ISER), used to enable an interrupt.
- Interrupt clear enable register (ICER), used to disable an interrupt.
- Interrupt clear pending register (ICPR), used to remove an interrupt from the pending queue.

8.6.1 Definitions of the NVIC struct and pointer

These registers are defined in the following struct for NVIC registers in `core_cm4.h`.

```
typedef struct {
  __IOM uint32_t ISER[8U];  // 0x000 (R/W)  Intr Set Enable
        uint32_t RESERVED0[24U];
  __IOM uint32_t ICER[8U];  // 0x080 (R/W)  Clear Enable
        uint32_t RESERVED1[24U];
  __IOM uint32_t ISPR[8U];  // 0x100 (R/W)  Set Pending
        uint32_t RESERVED2[24U];
  __IOM uint32_t ICPR[8U];  // 0x180 (R/W)  Clear Pending
        uint32_t RESERVED3[24U];
  __IOM uint32_t IABR[8U];  // 0x200 (R/W)  Active bit Reg
        uint32_t RESERVED4[56U];
  __IOM uint8_t  IP[240U];  // 0x300 (R/W)  Priority Reg
        uint32_t RESERVED5[644U];
  __OM  uint32_t STIR;      // 0xE00 ( /W)  SW Trigger
}  NVIC_Type;
```

Here, both `__IOM` and `__OM` are redefinitions of `volatile` for easy reading.

The following definition of pointers, including for NVIC, is also given in `core_cm4.h`.

```
#define SCS_BASE       (0xE000E000UL)  // Sys Control Space
#define CoreDebug_BASE (0xE000EDF0UL)  // Core Debug Base
#define SysTick_BASE   (SCS_BASE +  0x0010UL)  // SysTick
#define NVIC_BASE      (SCS_BASE +  0x0100UL)  // NVIC Base

#define CoreDebug ((CoreDebug_Type *)CoreDebug_BASE)
#define SysTick   ((SysTick_Type   *)SysTick_BASE  )
#define NVIC      ((NVIC_Type      *)NVIC_BASE     )
```

8.6.2 Direct access of NVIC registers

We have seen from a previous enum definition that the USART2 global interrupt has an IRQn being 38, as shown below

```
  USART2_IRQn           = 38,  // USART2 global Interrupt
```

We know that an element of ISER can hold 32 bits, each for an IRQn. As such, the index for the ISER register should be `IRQn / 32`:

- `IRQn / 32= 0` if IRQn is between 0 and 31; and
- `IRQn / 32= 1` if IRQn is between 32 and 63.

On the other hand, the bit location in a specific ISER register should be `IRQn % 32`:

- `IRQn % 32= IRQn` if IRQn is between 0 and 31; and
- `IRQn % 32= IRQn - 32` if IRQn is between 32 and 63.

Putting them together, we have

```
NVIC->ISER[IRQn / 32] = 1U << (IRQn % 32);
```

Considering the fact that $2^5 = 32$, we have an equivalent expression, which is potentially more efficient:

```
NVIC->ISER[IRQn >> 5] = 1U << (IRQn & 0x1F);
```

We can access ICER (Interrupt Clear Enable Register), ISPR (Interrupt Set Pending Register) and ICPR (Interrupt Clear Pending Register) in the same way.

8.7 ISR and callback function

8.7.1 Callback function of the SysTick timer

We have seen the **vector table** before, where each ISR is defined in the table.

Here, we reproduce the code to illustrate how it works:

```
    .word  SysTick_Handler
```

In the above code, `SysTick_Handler` is the name of the ISR, which is defined in ASM, in the same source file `startup_stm32g431kbtx.s` after the vector table, as

```
    .weak  SysTick_Handler
    .thumb_set SysTick_Handler,Default_Handler
```

This `.weak` modifier for `SysTick_Handler` is actually only a placeholder, which is needed so the program can compile. The true ISR is defined in `stm32g4xx_it.c` which is generated by CubeMX automatically as:

```
void SysTick_Handler(void) {
    /* USER CODE BEGIN SysTick_IRQn 0 */

    /* USER CODE END SysTick_IRQn 0 */
    HAL_IncTick();
    /* USER CODE BEGIN SysTick_IRQn 1 */

    /* USER CODE END SysTick_IRQn 1 */
}
```

If we look at the definition of `HAL_IncTick()`, in the generated code `stm32g4xx.hal.c`, we will see that it is defined as

```
__weak void HAL_IncTick(void) {
    uwTick += uwTickFreq;
}
```

The `__weak` modifier is used to indicate the current function modified by `__weak` can be replaced by a **normal** function with the same name defined by the user in a user source file.

The callback function we have used for the demo project in Lctr 7 is given below:

```c
// Callback function for SysTick timer:
extern __IO uint32_t uwTick;
extern uint32_t uwTickFreq;
int time_to_toggle = 2000; // Two seconds

void HAL_IncTick(void) {
    uwTick += uwTickFreq;
    if (uwTick % time_to_toggle == 0) {
        mp_toggle_LEDs();
    }
}
```

We can see that we have kept the same original code for the updating of the SysTick timer. We have also added new user code to toggle LEDs every two seconds.

8.7.2 A callback function for UART working in the interrupt mode

Below is the code in the super loop of the main program called by `mp_main()`:

```c
int dly_val_4_blink = 200;
uint8_t uart2_rxBuffer[10];

extern char msg_pmpt[];
extern int min_delay, max_delay;

void mp_app(void) {
    printf("\ndg2f App starts here:---\n");
    printf(msg_pmpt, min_delay, max_delay);

    HAL_UART_Receive_IT(&huart2, uart2_rxBuffer, 1);

    while (1) {
        HAL_GPIO_TogglePin(LED4_GPIO_Port, LED4_Pin);
        HAL_Delay(dly_val_4_blink); // Value can change
    }
}
```

Here, `uart2_rxBuffer` is defined with 10 elements, large enough for our application, where we only use the buffer to receive ONE byte in the `HAL_UART_Receive_IT` function.

The reason for receiving one byte is that we want to handle each byte directly in the callback function, which is defined below:

```c
#define CRG_RETURN 13   // ASCII code for carriage return

extern int dly_val_4_blink;
extern uint8_t uart2_rxBuffer[];

char msg_pmpt[] = "\nInput a number between %d and %d for
   delay:\n";
char err_pmpt[] = "\nInput out of range.";
char cmd_core[] = "               ";
int min_delay = 50;
int max_delay = 500;
int i = 0;
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart) {
    uint8_t new_input = uart2_rxBuffer[0];
    if (new_input >= '0' && new_input <= '9') {
        cmd_core[i++] = new_input;
        HAL_UART_Transmit(&huart2, &new_input, 1, 1000);
    } else if (new_input == CRG_RETURN) {
        cmd_core[i] = 0;
        int temp = atoi(cmd_core);
        if (temp >= min_delay && temp <= max_delay) {
            dly_val_4_blink = temp;
        } else {
            printf("%s", err_pmpt);
        }
        i = 0;
        printf(msg_pmpt, min_delay, max_delay);
    }

    HAL_UART_Receive_IT(&huart2, uart2_rxBuffer, 1);
}
```

In this callback function, we read a number between `min_delay` and `max_delay` from the UART, char by char, from the user. This number will determine the blinking rate. Note that at the end of the callback function, we need to restart receiver to receive the next byte.

8.8 Additional reading

8.8.1 USART setup for interrupt operations in CubeMX

In the **Configuration** field of the **USART2 Mode and Configuration** pane, click the **NVIC Settings** tab and check the **Enabled** checkbox for **USART2 global interrupt / USART2 wakeup . . . .**

8.8.2 C functions for UART interrupt operations

There are two functions related to UART receiving with interrupt:

- `HAL_UART_Receive_IT(&huart2, UART2_rxBuffer, bf_size)` for starting receiving using a buffer.
- `HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)` for defining the callback function by the user.

Note that we need to restart the receiving inside the callback function, as shown below:

```
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart) {
    HAL_UART_Transmit(&huart2, UART2_rxBuffer, 26, 100);
    HAL_UART_Receive_IT(&huart2, UART2_rxBuffer, 26);
}
```

8.8.3 MP core registers used for handling the interrupts

8.8.4 Two modes of running code

The MP core has the following two modes for running code:

- Thread mode. Used to execute the application software. MSP will be used for all our programs. PSP can also be used if we use a real-time operating system.
- Handler mode. Used to handle exceptions and interrupts. MSP is used in this mode.